

COT5520: COMPUTATIONAL GEOMETRY

Homework # 4

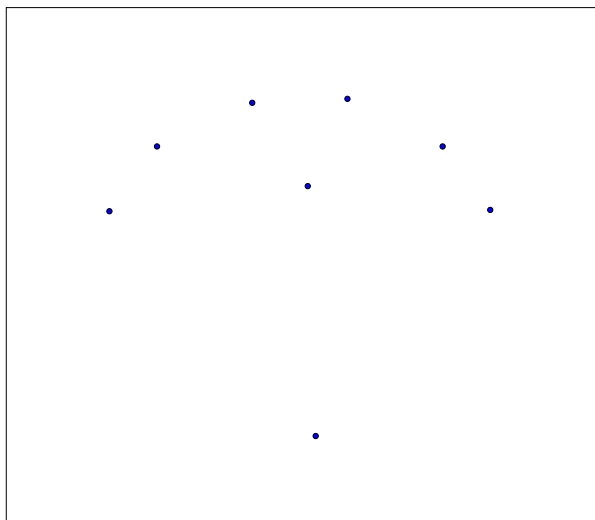
Due date: March 23, 2026, Monday

Your solutions should be concise, but complete, and typed (or handwritten clearly). Feel free to consult textbooks, journal and conference papers and also each other, but write the solutions yourself and cite your sources. Answer **a total of five questions (at least two on Voronoi Diagrams Q1-3, at least two on Delaunay Triangulations Q4-6)** of the following seven questions. Each problem is worth 20 pts.

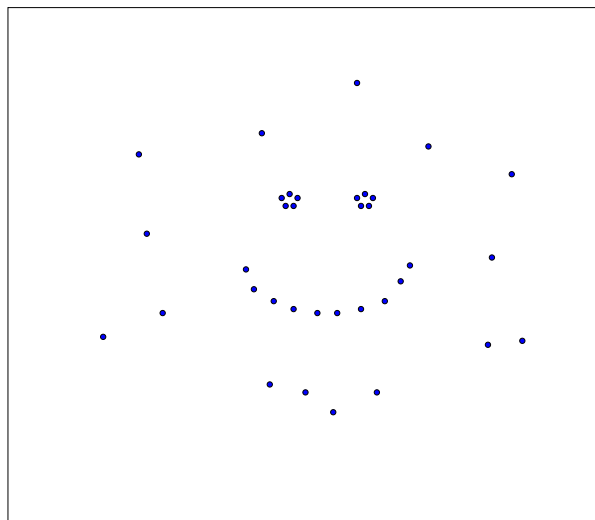
1. An order- k Voronoi diagram partitions the plane according to the k closest sites. A Voronoi cell in this generalization is the region of points which have the same k nearest sites. The order-1 Voronoi diagram is nothing more than the standard Voronoi diagram. The order- $(n-1)$ Voronoi diagram is also called the furthest neighbor Voronoi diagram, because the Voronoi cell of a point p_i now is the region of points for which p_i is the furthest site.

(a) Draw the order-2 Voronoi diagram of the following point set (a).

(b) Draw the furthest neighbor Voronoi diagram of the following point set (b).



(a)



(b)

2. Design, analyze, and implement an algorithm for computing the furthest neighbor Voronoi Diagram (see the previous problem for its definition) of a given point set. Perform an experimental study verifying the correctness and the running time of your algorithm.
3. * [More challenging than usual] Suppose that we are given a subdivision of the plane into n convex regions. We suspect that this division is a Voronoi diagram, but we do not know the sites. Design and analyze an algorithm that finds a set of n point sites whose Voronoi diagram is exactly the given subdivision, if such a set exists. Assume that the degree of the vertices in the subdivision are all three, which means no four Voronoi sites are co-circular.

4. (a) The degree of a point in a triangulation is the number of edges incident to it. Give an example of a set of n points in the plane such that, no matter how the set is triangulated, there is always a point whose degree is $n - 1$.
- (b) The randomized incremental algorithm discussed in class computes the Delaunay triangulation of a set of n points in $O(n \log n)$ expected time. Prove that the worst-case running time of the algorithm is $\Omega(n^2)$. Your construction should be generic, working for any n .

[Hint: While these are two different exercises in Chapter 9, your solution in part (a) should inspire the construction in part (b).]

5. Recall that the Delaunay Triangulation (DT) of a set of points in the plane is a maximally connected planar straight line graph. Other interesting and useful planar graphs (not maximal) include Euclidean Minimum Spanning Tree (EMST), Relative Neighborhood Graph (RNG), and Gabriel Graph (GG). (Check Chapter 9 in your textbook for their definitions). Prove that $\text{EMST} \subseteq \text{RNG} \subseteq \text{GG} \subseteq \text{DT}$.
6. The *weight* of a triangulation is the sum of the length of all edges of the triangulation. The *minimum weight triangulation* of a set S of n points is one whose weight is minimum among all triangulations of S . The *minmax angle triangulation* of S is one whose largest angle is minimized.
 - (a) Prove by example that the minimum weight triangulation is not the same as the Delaunay triangulation. Prove that the ratio of the weight of the Delaunay triangulation to the weight of the minimum weight triangulation is bounded by $O(n)$. Show (by a generic example, for any n) that your bound is tight.
 - (b) Prove by example that the minmax angle triangulation is not the same as the Delaunay triangulation. Prove that the ratio of the largest angle of the Delaunay triangulation to the largest angle of the minmax angle triangulation is bounded from above by 2. Show that this bound is (almost) tight.
7. Implement (at least) two of the Delaunay triangulation algorithms discussed in class (edge-flip, randomized incremental, or plane-sweep Voronoi). Run experiments with data sets of various size (say in the range 100-10,000). Does the running time of the implemented algorithm behave as predicted with our analysis? (For this problem you may use libraries for data structures or numerical predicates, but you should implement the main routine yourself.